

AI-Based Classroom Photo Attendance Management System: CampusEYE

1st Md. Arebi Sarker

Dept. of Electrical & Computer Engineering
North South University
Dhaka, Bangladesh
arebi.sarker@northsouth.edu

2nd Md. Rizvi Ahmed

Dept. of Electrical & Computer Engineering
North South University
Dhaka, Bangladesh
rizvi.safin@northsouth.edu

3rd Safayat Ibrahim

Dept. of Electrical & Computer Engineering
North South University
Dhaka, Bangladesh
safayat.ibrahim@northsouth.edu

4th Rafiqul Islam

Dept. of Electrical & Computer Engineering
North South University
Dhaka, Bangladesh
rafiqul.islam12@northsouth.edu

Abstract—This paper presents the second week’s progress in developing an automated attendance system based on face detection and recognition. The core functionality for Week 2 focuses on detecting faces in images, generating discriminative embeddings, and matching them against a pre-enrolled student database. We implement a detection module using OpenCV’s Haar cascade classifier and a recognition module leveraging DeepFace with the Facenet model to produce 128-dimensional embeddings. The system computes cosine distances between embeddings and classifies a face as known if the distance falls below a tunable threshold. We discuss the integration of these components into a full pipeline that processes images in real time, and we evaluate performance through a series of tests. Our results show that with an appropriate threshold, the system achieves reliable recognition, while also highlighting the need for further tuning in challenging conditions.

I. INTRODUCTION

Automated attendance management reduces manual effort and errors in educational institutions. The system under development captures faces from a camera, recognises registered students, and records their presence. In Week 1, we established the environment, designed the SQLite database, and implemented student enrolment (storage of averaged face embeddings). Week 2 extends this foundation by implementing the detection and recognition modules, enabling the system to identify individuals from an image.

The primary objectives of Week 2 (tasks W2-T1 to W2-T7) are:

- Detect all faces in an image and return bounding boxes.
- Generate face embeddings for each detected face.
- Compare embeddings against stored student embeddings using a distance metric.
- Determine the best match and classify as known/unknown based on a threshold.
- Combine detection and recognition into a single pipeline.
- Experiment with threshold values to balance false positives and negatives.

This report details the methodology, implementation, and test results for each task, concluding with a discussion of performance and future improvements.

II. METHODOLOGY

A. Face Detection

We employ OpenCV’s pre-trained Haar cascade classifier for frontal face detection. The classifier is fast and works well for near-frontal faces, which is sufficient for a classroom attendance scenario. The detection function `detect_faces()` converts the input image to grayscale and applies the cascade with parameters tuned for minimal false positives (scale factor 1.1, minNeighbors 5, minimum face size 80×80 pixels). This returns a list of bounding boxes (x, y, w, h) .

B. Face Recognition

For recognition, we use the DeepFace library with the Facenet model, which outputs a 128-dimensional embedding vector. Facenet has proven effective in face verification and recognition tasks. The embeddings are L2-normalised, allowing cosine distance as a similarity measure. Given a test embedding $\mathbf{e}_t$ and a stored embedding $\mathbf{e}_s$, the cosine distance is computed as:

$$d(\mathbf{e}_t, \mathbf{e}_s) = 1 - \frac{\mathbf{e}_t \cdot \mathbf{e}_s}{\|\mathbf{e}_t\| \|\mathbf{e}_s\|}.$$

The system finds the stored student with the smallest distance. If this distance is below a threshold τ , the face is considered a match to that student; otherwise, it is labelled “Unknown”.

C. Threshold Tuning

The choice of τ critically affects system performance. A low threshold reduces false positives (misclassifying unknown faces as known) but may increase false negatives (failing to recognise a known student). Conversely, a high threshold increases true positives but also false positives. We experiment

with values from 0.1 to 0.8 in steps of 0.05 and evaluate accuracy on a small validation set of positive (same person) and negative (different persons) pairs to select an optimal value.

D. Pipeline Integration

The detection and recognition modules are combined into a single pipeline: an input image is first processed by `detect_faces()` to obtain all face regions; each region is cropped and passed to the embedding generator; the resulting embeddings are then compared against the enrolled students using `recognize_face()`. The pipeline returns a list of recognised identities (or “Unknown”) for each detected face.

III. IMPLEMENTATION DETAILS

All code is written in Python 3.9+ and relies on the following libraries:

- OpenCV (cv2) for image I/O and Haar cascade.
- DeepFace for embedding extraction.
- NumPy and SciPy for numerical operations and distance computation.
- SQLite3 for database interactions.

The project structure follows modular design:

- `detection.py`: contains `detect_faces()`, `draw_face_boxes()`, and `extract_face()`.
- `recognition.py`: implements `get_face_embedding()`, `compare_faces()`, `recognize_face()`, and `batch_recognize()`.
- `config.py`: centralises configuration constants, including `DEEPFACE_MODEL="Facenet"`, `DEEPFACE_DETECTOR="opencv"`, and `RECOGNITION_THRESHOLD=0.6` (initial value).
- `test_week2.py`: provides interactive test scripts for each task.

A critical design decision was to pre-load all enrolled student embeddings into memory at the start of each session, thereby avoiding repeated database queries and improving runtime performance.

IV. EXPERIMENTAL RESULTS

We conducted a series of tests to validate each component.

A. Face Detection Test

Using a standard webcam, we captured a single frame containing a face. The detection function successfully returned one bounding box with coordinates matching the face position. When tested with a group photo (downloaded from the internet) containing five faces, the detector identified all five, albeit with one false positive on a background object. The false positive rate was acceptable for the current scope.

B. Recognition Test

With five enrolled students, we captured a test face and generated its embedding. The system compared this embedding against all stored embeddings and returned the correct student with a cosine distance of 0.23 (below the initial threshold of 0.6). For an unknown face (not in the database), the distance to the nearest enrolled student was 0.71, leading to a correct “Unknown” classification.

C. Full Pipeline Test

The integrated pipeline was tested on a live webcam feed. When a single face was present, it correctly identified the student and displayed their name. When multiple faces appeared (e.g., two students), both were recognised correctly. The processing time averaged 0.8 seconds per frame, which is acceptable for a prototype.

V. DISCUSSION

The implemented detection and recognition modules successfully meet the Week 2 requirements. The choice of Haar cascade, while fast, occasionally misses profiles or faces with extreme angles; this could be improved in later weeks by integrating a deep learning-based detector (e.g., RetinaFace) provided by DeepFace.

The threshold tuning experiment highlighted the importance of selecting a threshold that balances precision and recall. Our chosen value of 0.4 works well for our initial dataset, but it may need recalibration as the database grows or as lighting conditions change.

The pipeline performs adequately, but the embedding generation step (calling DeepFace) is the main bottleneck. We can mitigate this by downscaling images before detection or by using a lighter model.

VI. CONCLUSION AND FUTURE WORK

We have successfully implemented and tested the face detection and recognition components for our automated attendance system. The modules are integrated into a pipeline that can process images in real time. The system currently achieves high recognition accuracy when the threshold is properly tuned.